LangÇ#

Manual de Referência da Linguagem

Versão 1.0 – Compiladores / UNESC

Grupo: Gabriel Goulart de Souza, Luís Gabriel, Mateus Mariot, Victor Gaz e Karen Stackoskci.

1. Introdução

LangÇ# é uma linguagem de programação imperativa de propósito educacional, criada para a disciplina de Compiladores do curso de Ciência da Computação da UNESC. Ela possui tipagem estática, suporte a funções com parâmetros, estruturas de controle de fluxo e operações aritméticas e relacionais.

A linguagem é estruturada em **funções**, onde cada função possui um tipo de retorno, uma lista de parâmetros e um bloco de comandos. O ponto de entrada do programa é a função `main`.

2. Estrutura Geral

Todo programa em LangÇ# é composto por uma ou mais funções. Não há um bloco global separado — tudo reside dentro de funções. Exemplo mínimo:

```
int main() {
    return 0;
}
```

Uma função mais completa, com parâmetros e variáveis locais:

```
float media(float a, float b) {
    float resultado;
    resultado = (a + b) / 2;
    return resultado;
}
```

3. Tipos de Dados

LangÇ# oferece dois tipos primitivos:

Tipo	Descrição	Exemplos
int	Números inteiros	0, 42, -7, 1000
float	Números de ponto flutuante (decimais)	1.5, 3.14, 0.001

> ⚠ Não há conversão implícita entre int e float. Certifique-se de usar o tipo correto para cada variável.

4. Declarações

4.1 Variáveis Locais

Variáveis devem ser declaradas no início de um bloco, antes de qualquer comando. A sintaxe é:

```
<tipo>  <identificador>  ;
```

Exemplos:

```
int main() {
    int contador;
    int total;
    float taxa;
    ç# ... comandos ...
    return 0;
}
```

4.2 Funções

A declaração de uma função segue o formato:

```
<tipo_retorno>  <nome>  ( <parametros> )  <bloco>
```

Parâmetros são separados por vírgula e cada um precisa de tipo explícito:

```
int somar(int x, int y) {
    return x + y;
}

float potencia(float base, int exp) {
    ç# ...
    return base;
}
```

5. Comandos

5.1 Atribuição

A atribuição usa o símbolo = (simples). A variável deve ter sido declarada previamente.

```
int main() {
    int x;
    int y;
    x = 10;
    y = x + 5;
    return 0;
}
```

5.2 print

Exibe o valor de uma expressão no terminal. Aceita qualquer expressão válida como argumento.

```
int main() {
    int n;
    n = 42;
    print(n);           ç# exibe: 42
    print(n * 2);       ç# exibe: 84
    print(n + 1);       ç# exibe: 43
    return 0;
}
```

5.3 return

Encerra a execução da função e retorna um valor ao chamador.

```
int quadrado(int n) {
    return n * n;
}
```

5.4 if / else

Estrutura de decisão. O `else` é opcional.

```
ç# Sem else
```

```
if (x > 0) {
    print(x);
}

ç# Com else
if (nota >= 6) {
    print(1);    ç# aprovado
} else {
    print(0);    ç# reprovado
}
```

É possível aninhar if/else para múltiplas condições:

```
if (n < 0) {
    print(0);    ç# negativo
} else {
    if (n == 0) {
        print(1);    ç# zero
    } else {
        print(2);    ç# positivo
    }
}
```

5.5 while

Executa um bloco repetidamente enquanto a condição for verdadeira.

```
int main() {
    int i;
    i = 1;
    while (i <= 10) {
        print(i);
        i = i + 1;
    }
    return 0;
}
ç# Saída: 1  2  3  4  5  6  7  8  9  10
```

5.6 Chamada de Função

Funções são chamadas pelo nome seguido de parênteses com os argumentos. O valor retornado pode ser usado em expressões.

```
int dobro(int n) {
    return n * 2;
}

int main() {
    int r;
    r = dobro(5);     ç# r = 10
    print(r);
    print(dobro(3));  ç# imprime diretamente: 6
    return 0;
```

```
}
```

5.7 Bloco

Um bloco é delimitado por `{` e `}`. Pode aparecer como corpo de uma função, de um `if`, de um `while` ou de forma independente como comando.

```
{
    int tmp;
    tmp = a;
    a = b;
    b = tmp;
}
```

6. Expressões e Operadores

6.1 Operadores Aritméticos

Operador	Operação	Exemplo	Resultado
+	Adição	3 + 4	7
-	Subtração	10 - 3	7
*	Multiplicação	4 * 5	20
/	Divisão	9 / 2	4 (int)

6.2 Operadores Relacionais

Operador	Significado	Exemplo
==	Igual a	x == 0
!=	Diferente de	x != y
<	Menor que	i < 10
>	Maior que	n > 100
<=	Menor ou igual	a <= b
>=	Maior ou igual	nota >= 6

6.3 Precedência

A precedência segue a ordem matemática convencional:

Prioridade	Operadores	Associatividade
1 (maior)	*, /	Esquerda → Direita
2	+, -	Esquerda → Direita
3 (menor)	==, !=, <, >, <=, >=	Esquerda → Direita

Use parênteses para forçar outra ordem:

```
int x;
x = (2 + 3) * 4;     ç# 20, não 14
```

7. Comentários

LangÇ# suporta dois estilos de comentário:

Comentário de linha

Tudo após ç# até o final da linha é ignorado pelo compilador.

```
int x;     ç# esta é uma variável inteira
x = 0;     ç# inicializa com zero
```

Comentário de bloco

Tudo entre ç@ e @ç é ignorado, mesmo que abranja múltiplas linhas.

```
ç@
  Calcula a soma dos n primeiros naturais.
  Parâmetro: n >= 0
  Retorno:   n*(n+1)/2
@ç
int somaAteN(int n) {
    return n * (n + 1) / 2;
}
```

8. Regras Léxicas

8.1 Identificadores

Um identificador válido deve:

- Começar com uma letra (a-z, A-Z) ou sublinhado (_)
- Conter apenas letras, dígitos e sublinhado
- Ter no máximo 64 caracteres

• Não coincidir com palavra reservada

Válido ✓	Inválido ✗	Motivo
contador	1contador	Inicia com dígito
_valor	val ue	Contém espaço
mediaFinal	médiaFinal	Contém acento
x		—

8.2 Literais Numéricos

• **Inteiros (num):** sequência de dígitos. Ex.: `0 42 9999`

• **Reais (num):** dígitos, ponto, dígitos. Ex.: `1.5 3.14 0.001`

8.3 Palavras Reservadas

Palavra	Uso
int	Tipo inteiro
float	Tipo ponto flutuante
if	Decisão condicional
else	Ramo alternativo do if
while	Repetição com condição
print	Saída no terminal
return	Retorno de função

9. Erros Léxicos Comuns

Erro	Exemplo	Descrição
Identificador inválido	2x = 1;	Inicia com dígito
Identificador com acento	médiaFinal = 0;	Caractere especial
Identificador muito longo	aaa…aaa (65+);	Acima de 64 chars
Operador desconhecido	x $ y;	Símbolo não suportado

10. Exemplo Completo

Programa que calcula o fatorial de números de 1 a 5 usando recursão e exibe se cada um é par ou ímpar:

```
ç# Retorna o fatorial de n (n >= 0)
int fatorial(int n) {
    int resultado;
    if (n <= 1) {
        resultado = 1;
        return resultado;
    }
    resultado = n * fatorial(n - 1);
    return resultado;
}

ç# Retorna 0 se x for par, 1 se for ímpar
int ehImpar(int x) {
    int resto;
    resto = x - (x / 2) * 2;
    return resto;
}

int main() {
    int i;
    int fat;
    i = 1;
    while (i <= 5) {
        fat = fatorial(i);
        print(fat);
        print(ehImpar(fat));
        i = i + 1;
    }
    return 0;
}
```

Saída esperada (fatorial | paridade):

```
1   1
2   0
6   0
24  0
120 0
```

Apêndice A — Gramática Formal

A gramática completa de LangÇ# em notação BNF:

```
<Program>      ::= <FunctionList>
<FunctionList> ::= <Function> <FunctionList'>
<FunctionList'>::= <Function> <FunctionList'> | ε
```

```
<Function>     ::= <Type> id ( <ParamListOpt> ) <Block>
<ParamListOpt> ::= <ParamList> | ε
<ParamList>    ::= <Param> <ParamList'>
<ParamList'>   ::= , <Param> <ParamList'> | ε
<Param>        ::= <Type> id
<Block>        ::= { <DeclListOpt> <StmtListOpt> }
<DeclListOpt>  ::= <DeclList> | ε
<DeclList>     ::= <VarDecl> <DeclList'>
<DeclList'>    ::= <VarDecl> <DeclList'> | ε
<VarDecl>      ::= <Type> id ;
<StmtListOpt>  ::= <StmtList> | ε
<StmtList>     ::= <Stmt> <StmtList'>
<StmtList'>    ::= <Stmt> <StmtList'> | ε
<Stmt>         ::= <AssignStmt> | <IfStmt> | <WhileStmt>
                 | <PrintStmt> | <ReturnStmt> | <Block>
<AssignStmt>   ::= id = <Expr> ;
<ReturnStmt>   ::= return <Expr> ;
<PrintStmt>    ::= print ( <Expr> ) ;
<IfStmt>       ::= if ( <Expr> ) <Stmt> <ElsePart>
<ElsePart>     ::= else <Stmt> | ε
<WhileStmt>    ::= while ( <Expr> ) <Stmt>
<Expr>         ::= <RelExpr>
<RelExpr>      ::= <AddExpr> <RelExpr'>
<RelExpr'>     ::= <RelOp> <AddExpr> | ε
<RelOp>        ::= == | != | < | > | <= | >=
<AddExpr>      ::= <MulExpr> <AddExpr'>
<AddExpr'>     ::= + <MulExpr> <AddExpr'>
                 | - <MulExpr> <AddExpr'> | ε
<MulExpr>      ::= <Factor> <MulExpr'>
<MulExpr'>     ::= * <Factor> <MulExpr'>
                 | / <Factor> <MulExpr'> | ε
<Factor>       ::= ( <Expr> ) | id <FactorTail> | num
<FactorTail>   ::= ( <ArgListOpt> ) | ε
<ArgListOpt>   ::= <ArgList> | ε
<ArgList>      ::= <Expr> <ArgList'>
<ArgList'>     ::= , <Expr> <ArgList'> | ε
<Type>         ::= int | float
```